Project Proposal — Sevai Setu FOR REVIEW

சேவை சேது · Track 1 — "Build with AI: Code for Communities" (Google Cloud)

Prepared for: Tech Lead review & approval to proceed · **Prepared by:** [Your name] · **Date:** 23 June 2026
Track: People's Priorities — AI for Constituency Development Planning · **Demo target:** Dharmapuri Lok Sabha, Tamil Nadu
Submission deadline: 8 July 2026 · **Demo Day:** 23 July 2026, New Delhi

1. Executive summary

We are building **Sevai Setu** — an AI decision-support system for Members of Parliament that turns scattered, multilingual citizen demands and fragmented public data into **a clear, defensible action plan**.

The differentiating insight: an MP's own fund (**MPLADS ≈ ₹5 crore/year**) is tiny next to the **hundreds of crores of existing government schemes** (housing, water, jobs, pensions) already meant for their constituents. **Most citizen needs are unfilled *entitlements*, not requests for new money.** So instead of "yet another complaint dashboard," our engine:

1. Routes each citizen need to the **scheme that should already cover it**.
2. Shows, village-by-village, **where scheme money is failing to land** (the "Constituency X-Ray").
3. Recommends **(A)** unlock existing entitlements for ₹0 of the MP's fund, and **(B)** spend the ₹5 cr only on the true gaps nothing else covers — fairly, including villages that never complain.

Status: concept finalised, design spec written, **clickable HTML prototype already built**, intake approach decided. **We are seeking approval to begin the 15-day build.**

2. The problem (Track 1, as stated)

> *MPs receive development requests through public meetings, letters, social media, grievance portals, and direct representations — while local development plans contain dozens of competing proposed projects. There's no objective way to consolidate citizen feedback, spot recurring needs, and weigh competing proposals against real demand.*

Our sharper framing: decisions favour whoever **complains loudest**; they are ad-hoc and politically exposed; **silent, poor, remote areas are systematically ignored**; and the MP's scarce fund is often spent on things **other schemes should already cover** while genuine entitlement-delivery failures go unaddressed.

3. Our solution

"We don't help an MP spend ₹5 crore. We help them unlock the ₹500 crore of schemes already meant for their people — and spend their own ₹5 crore only on the gaps nothing else covers."

The engine (5 steps)

1. **Understand** — Tamil/English voice & text → structured demand record {need, sector, village→LGD code, urgency, beneficiaries} via Gemini + Speech-to-Text + Translation.

2. **Match to scheme** — classify each need to the responsible scheme/fund (e.g. housing → PMAY-G) vs. "no scheme covers this → MPLADS candidate."
3. **Reconcile with ground truth** — join real public datasets; compute a transparent Need/Gap score: gap = eligible/needy population − population already covered.
4. **Detect the silent gap** — flag villages with high measured need but zero petitions.
5. **Recommend two tracks** — Track A (unlock entitlements, ₹0) + Track B (budget-optimised use of ₹5 cr on true gaps), each with auto-generated justification.

> *Design principle: the priority score is a **transparent weighted formula, not a black-box ML model** — government will only deploy what it can explain and audit.*

Why this is differentiated (honest)

Most teams build complaint-collection + categorisation + heatmap + ranking. We do that **plus** the scheme-convergence layer — requiring real understanding of how Indian governance/money works and integrating real scheme-coverage data. **The moat is domain insight + data execution, not a single feature.** We deliberately **avoid fabricated "impact prediction" numbers** (e.g. "reduces dropout by 12%") — unprovable and a knowledgeable judge/MP will reject them. All our figures are **computable and real** (people served, eligible-but-unserved counts, distance to nearest facility).

4. Key features (build scope)

Feature	Notes
Multilingual (Tamil) voice/text intake	Speech-to-Text + Translation + Gemini
Scheme-matching & entitlement routing	The core differentiator
Constituency X-Ray (village coverage heatmap)	Hero screen, Maps Platform
Transparent Need/Gap scoring	Auditable formula
Silent-village (equity) detection	Need-weighted, not complaint-weighted
Two-track recommendations (Unlock / Spend ₹5 cr)	OR-Tools budget optimisation
Auto-drafted official letters & memos	Gemini
Satellite "proof" for 2–3 hero villages	Earth Engine (depth, not breadth)
(Stretch) Voice "ask your constituency" assistant	Gemini over BigQuery — demo flourish

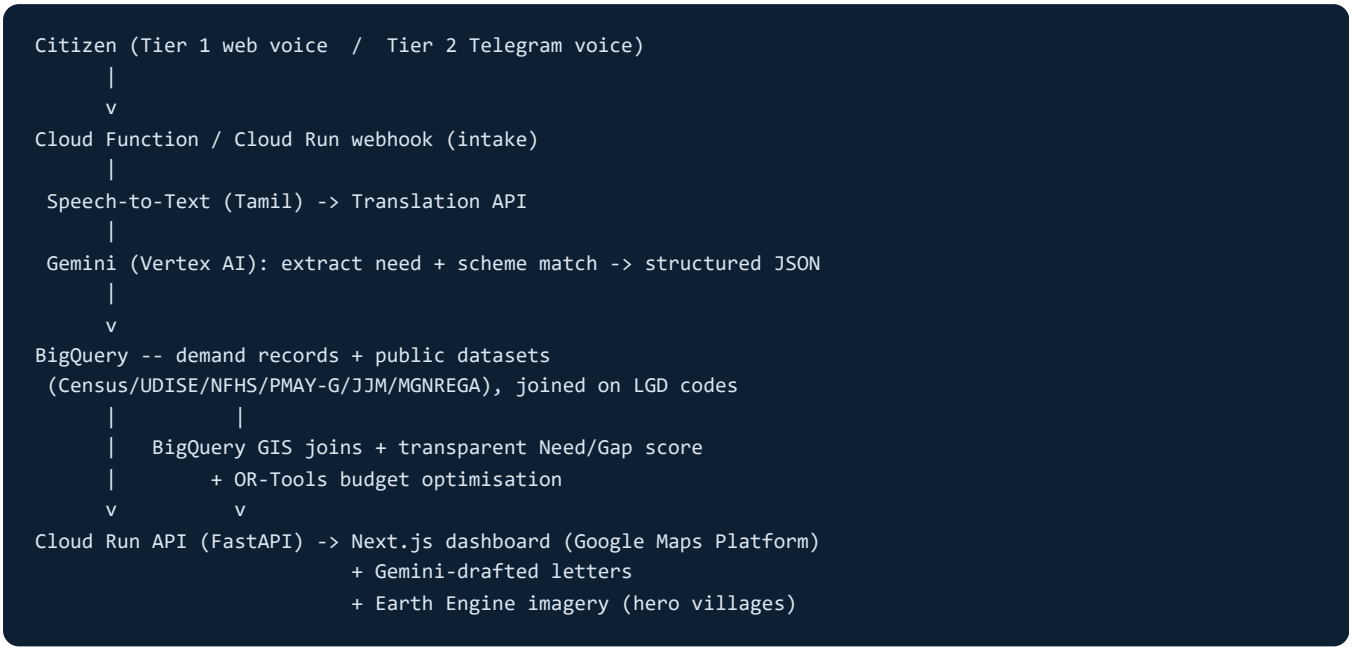
5. Intake approach

Both feed the **same backend pipeline** (no duplicate work):

- **Tier 1 — own web + voice page (reliable backbone):** Tamil mic/upload; DB pre-loaded with 200–500 synthesised realistic Tamil petitions mapped to real Dharmapuri villages. 100% under our control.
- **Tier 2 — Telegram bot (live "wow"):** free, no approval, accepts Tamil voice notes; a judge can message it live and see the demand appear on the dashboard in seconds.

- **WhatsApp: out of scope** (Business API approval too slow). Pitch line: "intake is channel-agnostic; built to plug into WhatsApp / CM Helpline in production."

6. Architecture (Google Cloud native)



Stack: Next.js · FastAPI on Cloud Run · Cloud Functions · Vertex AI / Gemini · Cloud Speech-to-Text · Translation API · BigQuery + BigQuery GIS · OR-Tools · Earth Engine · Google Maps Platform · Firebase. All serverless; runs on hackathon Google Cloud credits.

7. Data plan (real Tamil Nadu sources)

Layer	Source	Resolution
Geographic join key	LGD (Local Government Directory)	village/ward
Demographics & amenities	Census 2011 (TN)	village
Education need	UDISE+ (TN)	school/village
Health need	NFHS-5 (TN) + PHC data	district
Housing coverage	PMAY-G MIS / AwaasSoft	village/block
Water coverage	Jal Jeevan "Har Ghar Jal" dashboard	village
Jobs/assets	MGNREGA MIS	panchayat
Anti-duplication	MPLADS portal (filter TN MP)	work-level
Satellite proof	Earth Engine / Bhuvan	image
Citizen petitions	Synthesised (real feeds not open)	—

Honesty notes (stated in pitch): petitions are synthesised; some scheme data is block/district-level only (best-available used); satellite verification deep for 2–3 hero villages, simulated for breadth.

8. Scope for the hackathon

In: one constituency (Dharmapuri); real Census + UDISE + 3–4 scheme datasets (PMAY-G, Jal Jeevan, MGNREGA + one pension/PM-Kisan); Tamil intake (Tier 1 + Tier 2); scheme-matching; X-Ray; transparent scoring; two-track recommendations; auto-letters; ₹5 cr optimiser; silent detection; 2–3 satellite hero villages.

Out (stated honestly): all 30+ schemes; live WhatsApp; statewide rollout; causal outcome/impact prediction.

9. Indicative 15-day plan

Days	Focus
1–3	Data acquisition + cleaning (LGD, Census, UDISE, scheme dashboards); BigQuery schema
4–6	Intake pipeline (Tier 1 web + Tier 2 Telegram) → Speech-to-Text → Gemini → BigQuery
5–8	Scheme-matching + Need/Gap scoring + silent detection (hardest data-join work — most time here)
7–10	Dashboard: X-Ray map + scheme-gap list + two-track screens
9–12	OR-Tools budget optimiser + auto-letters/memos; deploy to Cloud Run
12–13	Satellite hero villages + (stretch) voice assistant
13–14	Demo video, pitch deck, polish
15 (8 Jul)	**Submit** (source code, demo video, pitch deck, deployed link)

10. Production rollout (the "what's next?" answer)

- **Phase 1 — Pilot (90 days, one constituency):** data-sharing MoU with district admin; connect one real intake channel; office staff validate flagged gaps; measure petitions processed, ₹ entitlements unlocked, forgotten villages reached.
- **Phase 2 — Harden (3–6 mo):** privacy & security to government standard (consent, minimal PII, MeitY-empanelled/compliant cloud, GIGW); integrate CM Helpline 1100 / e-Sevai / CPGRAMS.
- **Phase 3 — Scale (6–18 mo):** roll out to TN's 39 MPs + 234 MLAs, then other states; natural owner = state e-gov agency (TNeGA) / District Planning Committee. Serverless ⇒ low run cost; scaling is configuration, not rebuild.

11. Risks & mitigations

Risk	Mitigation
Concept is graspable once described	Win on execution depth with real TN data; don't over-explain pre-submission
Hardest work = messy data joins (Tamil → LGD → scheme coverage)	Front-load it (days 1–8); most effort here, not UI
Scheme data resolution varies	Use best-available level; be transparent

"This is just convergence/DPC planning"	Rebuttal: "Yes — it fails because nobody joins the data at village level; we did."
Scope creep in 15 days	Strict in/out scope; depth over breadth

12. Team & resources needed

- **Team strengths:** frontend, backend/APIs, AI/ML & data, mobile/design (full-stack).
- **Lead:** Google Cloud Professional Architect (deployment & GCP services).
- **Needed:** Google Cloud credits (provided to shortlisted teams), Vertex AI / Gemini access, Maps Platform key, Telegram bot token (free).

13. Approval requested

We are asking the Tech Lead to approve:

☐ **1. Direction — proceed with Sevai Setu (scheme-convergence / entitlement-gap engine) as the Track 1 entry.**

☐ **2. Tech stack — Google Cloud native stack as in §6.**

☐ **3. Scope — the in/out scope in §8 (depth over breadth, no fabricated impact prediction).**

☐ **4. Intake — Tier 1 (own web/voice) + Tier 2 (Telegram bot).**

☐ **5. Go-ahead to start the detailed 15-day build.**

References in repo: prototype/index.html · docs/sevai-setu-design-spec.md · docs/build-with-ai-code-for-communities.md

---

Reviewer comments: _____

Decision: ☐ Approved ☐ Approved with changes ☐ Needs rework

Signature / Date: _____